Purpose

POL-0 — policy/ Purpose + boundaries

Piece A — Purpose

File: `Axion/libraries/policy/POL-0_purpose.md`

id: POL-0
library: policy
schema_version: 1.0.0
status: draft

POL-0 — policy/ Purpose + Boundaries

Purpose
`policy/` defines Axion's **global governance rules**:
- what "risk class" means (prototype vs prod vs compliance)
- what hard stops exist
- when overrides are allowed
- who can approve overrides
- how overrides expire
- how policy conflicts are resolved deterministically

This is where Axion decides when it can bend its own rules—without becoming non-deterministic.

What it governs (in scope)
- Risk class definitions and required thresholds (maturity floors, stricter gates)
- Override policy (who/when/why/expiry, approvals, auditability)
- Policy precedence and conflict resolution
- Policy enforcement points (which stages/gates consult policy)
- Policy sets (named bundles of policy rules bound to workspaces/projects)

What it does NOT govern (out of scope)
- How policy is *invoked* at runtime (hook plumbing) → `system/`
- Gate DSL grammar/evaluation → `gates/`
- Stage ordering, run manifest format → `orchestration/`
- Domain-specific content rules (knowledge/templates/standards/etc.) live in those libraries

Consumers
- Policy engine (evaluates requests from SYS hook points)
- Orchestrator (pause/stop decisions based on policy outcomes)
- Gate evaluator (override permission checks)

- Operator UI (approval workflows, policy views)

Determinism requirements
- Policy decisions must be a pure function of:
  - pinned policy set
  - explicit request context
  - recorded identity/roles
- Overrides must be recorded with stable IDs and expiries.

Piece B — Boundary checklist

File: `Axion/libraries/policy/POL-0_boundary_checklist.md`

id: POL-0a
library: policy
schema_version: 1.0.0
status: draft

POL-0a — Boundary Checklist

Belongs in `policy/` if it answers:
- “What does PROD/COMPLIANCE require?”
- “When can we override a gate or quota?”
- “Who can approve and what must be recorded?”
- “How do conflicting rules resolve?”

Does NOT belong in `policy/` if it answers:
- “How do we call policy?” (system)
- “How do we evaluate predicates?” (gates)
- “What stages exist?” (orchestration)

Risk classes model

POL-1 — Risk class definitions model

Piece A — Risk class model (concept)

File: `Axion/libraries/policy/POL-1_risk_class_model.md`

```
---
library: policy
id: POL-1
schema_version: 1.0.0
status: draft
---

# POL-1 — Risk Class Model

## Purpose
Risk classes define the baseline strictness of a run. They drive:
- minimum maturity thresholds (knowledge/standards)
- which gates are hard-stop vs warn vs optional
- whether external executors are allowed
- whether approvals are required for certain actions

## Core risk classes (minimum)
- PROTOTYPE: speed-first, lowest constraints
- PROD: reliability + security baseline
- COMPLIANCE: regulated/high-stakes constraints

## What risk classes control (minimum knobs)
- knowledge min_maturity
- standards strictness level
- template completeness strictness
- verification requirements (proof types required)
- override availability (what can be overridden)
```

Piece B — Risk class schema (JSON Schema)

File: `Axion/libraries/policy/schemas/risk_classes.v1.schema.json`

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "schema_version": "1.0.0",
```

```
  "id": "risk_classes.v1",
  "type": "object",
  "required": ["registry_id", "schema_version", "classes"],
  "properties": {
    "registry_id": { "type": "string", "minLength": 3 },
    "schema_version": { "type": "string", "minLength": 1 },
    "classes": {
      "type": "array",
      "minItems": 1,
      "items": { "$ref": "#/$defs/risk_class" }
    }
  },
  "$defs": {
    "risk_class": {
      "type": "object",
      "required": [
        "risk_class",
        "description",
        "thresholds",
        "gate_strictness",
        "executor_rules"
      ],
      "properties": {
        "risk_class": { "type": "string", "enum": ["PROTOTYPE", "PROD", "COMPLIANCE"] },
        "description": { "type": "string", "minLength": 5 },

        "thresholds": {
          "type": "object",
          "required": ["knowledge_min_maturity"],
          "properties": {
            "knowledge_min_maturity": { "type": "string", "enum": ["draft", "reviewed", "verified",
"golden"] },
            "standards_mode": { "type": "string", "enum": ["lenient", "strict", "locked"] },
            "template_completeness": { "type": "string", "enum": ["basic", "strict"] },
            "verification_level": { "type": "string", "enum": ["basic", "standard", "strict"] }
          },
          "additionalProperties": false
        },

        "gate_strictness": {
          "type": "object",
          "description": "Overrides default gate severities per risk class",
          "additionalProperties": { "type": "string", "enum": ["hard_stop", "pause", "warn"] }
        },
```

```
      "executor_rules": {
        "type": "object",
        "required": ["external_allowed"],
        "properties": {
          "external_allowed": { "type": "boolean" },
          "external_restrictions": {
            "type": "array",
            "items": { "type": "string" }
          }
        },
        "additionalProperties": false
      }
    },
    "additionalProperties": false
  }
},
"additionalProperties": false
}
```

Piece C — Default risk classes registry (starter)

File: `Axion/libraries/policy/registries/risk_classes.v1.json`

```
{
  "schema_version": "1.0.0",
  "registry_id": "risk_classes.v1",
  "classes": [
    {
      "risk_class": "PROTOTYPE",
      "description": "Fast iteration; minimal governance; suitable for internal prototypes.",
      "thresholds": {
        "knowledge_min_maturity": "draft",
        "standards_mode": "lenient",
        "template_completeness": "basic",
        "verification_level": "basic"
      },
      "gate_strictness": {},
      "executor_rules": {
        "external_allowed": true,
        "external_restrictions": ["no_internal_only_knowledge", "no_restricted_exports"]
      }
```

```
    },
    {
      "risk_class": "PROD",
      "description": "Production-quality; enforce reviewed knowledge and strict baseline gates.",
      "thresholds": {
        "knowledge_min_maturity": "reviewed",
        "standards_mode": "strict",
        "template_completeness": "strict",
        "verification_level": "standard"
      },
      "gate_strictness": {
        "G1_INTAKE_VALIDITY": "pause",
        "G2_CANONICAL_INTEGRITY": "hard_stop",
        "G7_VERIFICATION": "pause"
      },
      "executor_rules": {
        "external_allowed": true,
        "external_restrictions": ["no_internal_only_knowledge", "no_restricted_exports"]
      }
    },
    {
      "risk_class": "COMPLIANCE",
      "description": "High-stakes/regulated; enforce verified knowledge and locked standards
behavior.",
      "thresholds": {
        "knowledge_min_maturity": "verified",
        "standards_mode": "locked",
        "template_completeness": "strict",
        "verification_level": "strict"
      },
      "gate_strictness": {
        "G1_INTAKE_VALIDITY": "pause",
        "G2_CANONICAL_INTEGRITY": "hard_stop",
        "G7_VERIFICATION": "pause"
      },
      "executor_rules": {
        "external_allowed": false,
        "external_restrictions": ["internal_execution_only"]
      }
    }
  ]
}
```

Piece D — Determinism rules

File: `Axion/libraries/policy/POL-1_determinism_rules.md`

```
---
library: policy
id: POL-1a
schema_version: 1.0.0
status: draft
---

# POL-1a — Determinism Rules (Risk Class)

- risk_class is a closed enum (PROTOTYPE/PROD/COMPLIANCE).
- Threshold fields are enums, not free text.
- Risk class selection must be recorded in the run manifest at run start.
- Any changes to risk class definitions require a registry version bump.
```

Piece E — Validation checklist

File: `Axion/libraries/policy/POL-1_validation_checklist.md`

```
---
library: policy
id: POL-1b
schema_version: 1.0.0
status: draft
---

# POL-1b — Validation Checklist

- [ ] risk_classes registry validates against schema
- [ ] each class includes thresholds + gate_strictness + executor_rules
- [ ] knowledge_min_maturity is set for all classes
- [ ] risk_class values are from the closed enum set
```

Overrides model

POL-2 — Override policy model (who/when/how/expiry)

Piece A — Override model (concept)

File: `Axion/libraries/policy/POL-2_override_policy_model.md`

```
---
library: policy
id: POL-2
schema_version: 1.0.0
status: draft
---

# POL-2 — Override Policy Model

## Purpose
Overrides allow Axion to proceed when a hard rule would block progress, but only under
controlled conditions:
- explicit reason
- explicit approver identity/role
- explicit expiry
- recorded evidence
- reproducible record in run manifest + audit log

## Where overrides apply (minimum)
- Gate failures (GATE_OVERRIDE hook)
- Quota exceed decisions (QUOTA_CHECK hook)
- Pin/lock exceptions (PIN_RESOLUTION hook)
- Adapter exceptions (ADAPTER_SELECTION hook)
- Kit export exceptions (KIT_EXPORT hook)

## Non-negotiables
- Every override must expire.
- Every override must specify scope (what it applies to).
- Overrides must be replay-safe (recorded as immutable facts in the run).
```

Piece B — Override request schema (JSON Schema)

File: `Axion/libraries/policy/schemas/override_request.v1.schema.json`

```
{
```

```
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "schema_version": "1.0.0",
  "id": "override_request.v1",
  "type": "object",
  "required": [
    "override_request_id",
    "run_id",
    "hook_point",
    "scope",
    "reason",
    "requested_by",
    "requested_at"
  ],
  "properties": {
    "override_request_id": { "type": "string", "pattern": "^OVRREQ-[A-Z0-9]{6,}$" },
    "run_id": { "type": "string" },

    "hook_point": {
      "type": "string",
      "enum": ["GATE_OVERRIDE", "QUOTA_CHECK", "PIN_RESOLUTION",
"ADAPTER_SELECTION", "KIT_EXPORT"]
    },

    "scope": {
      "type": "object",
      "required": ["type"],
      "properties": {
        "type": { "type": "string", "enum": ["gate", "quota", "pin", "adapter", "export"] },

        "gate_id": { "type": "string" },
        "predicate_id": { "type": "string" },

        "quota_limit": { "type": "string" },

        "pin_target": { "type": "string" },

        "adapter_profile_id": { "type": "string" },

        "export_target": { "type": "string", "enum": ["internal", "external"] }
      },
      "additionalProperties": false
    },

    "reason": { "type": "string", "minLength": 8 },
```

```
    "requested_by": {
      "type": "object",
      "required": ["actor_id", "role"],
      "properties": {
        "actor_id": { "type": "string", "minLength": 2 },
        "role": { "type": "string", "enum": ["owner", "maintainer", "operator"] }
      },
      "additionalProperties": false
    },

    "requested_at": { "type": "string", "format": "date-time" },

    "evidence_refs": {
      "type": "array",
      "items": { "type": "string" },
      "description": "Pointers to supporting artifacts/reports (optional but recommended)"
    }
  },
  "additionalProperties": false
}
```

Piece C — Override decision schema (approval result)

File: `Axion/libraries/policy/schemas/override_decision.v1.schema.json`

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "schema_version": "1.0.0",
  "id": "override_decision.v1",
  "type": "object",
  "required": [
    "override_decision_id",
    "override_request_id",
    "run_id",
    "outcome",
    "approved_by",
    "decided_at",
    "expiry"
  ],
  "properties": {
    "override_decision_id": { "type": "string", "pattern": "^OVRDEC-[A-Z0-9]{6,}$" },
```

```
    "override_request_id": { "type": "string" },
    "run_id": { "type": "string" },

    "outcome": { "type": "string", "enum": ["approved", "denied"] },

    "approved_by": {
      "type": "object",
      "required": ["actor_id", "role"],
      "properties": {
        "actor_id": { "type": "string", "minLength": 2 },
        "role": { "type": "string", "enum": ["owner", "maintainer", "operator"] }
      },
      "additionalProperties": false
    },

    "conditions": {
      "type": "object",
      "description": "Optional constraints applied to approval (cap reduction, limited scope, etc.)",
      "additionalProperties": true
    },

    "decided_at": { "type": "string", "format": "date-time" },

    "expiry": {
      "type": "object",
      "required": ["expires_at"],
      "properties": { "expires_at": { "type": "string", "format": "date-time" } },
      "additionalProperties": false
    },

    "notes": { "type": "string" }
  },
  "additionalProperties": false
}
```

Piece D — Override rules (expiry + scope + recording)

File: `Axion/libraries/policy/POL-2_override_rules.md`

```
---
library: policy
id: POL-2a
```

schema_version: 1.0.0
status: draft

POL-2a — Override Rules

Expiry required
- Every approved override must include expiry.expires_at.
- Default max expiry windows (policy defaults):
  - PROTOTYPE: up to 30 days
  - PROD: up to 7 days
  - COMPLIANCE: up to 24 hours (or disallow entirely, per policy set)

Scope required
- Override scope.type must match hook_point.
- Gate overrides must specify gate_id (and optionally predicate_id).

Recording required
On approval:
- record override_decision in run manifest runtime.policy_decisions (or a dedicated overrides array)
- emit audit event (who/what/why/expiry)
- emit notification (approval granted)

No silent reuse
Overrides apply only to the run_id unless explicitly defined as a project-level policy exception (separate mechanism).

Piece E — Validation checklist

File: `Axion/libraries/policy/POL-2_validation_checklist.md`

---
library: policy
id: POL-2b
schema_version: 1.0.0
status: draft

POL-2b — Validation Checklist

- [ ] override_request validates against schema

- [ ] override_decision validates against schema
- [ ] approved overrides always include expires_at
- [ ] scope fields align with hook_point (gate -> gate_id, quota -> quota_limit, etc.)
- [ ] approved overrides are recorded in run manifest and audit log

Say **“next”** for **POL-3 Policy precedence + conflict resolution**.

Policy resolution

POL-3 — Policy precedence + conflict resolution (deterministic)

Piece A — Policy precedence model (concept)

File: `Axion/libraries/policy/POL-3_precedence_model.md`

---
library: policy
id: POL-3
schema_version: 1.0.0
status: draft

POL-3 — Policy Precedence + Conflict Resolution

Purpose
Policies may come from multiple layers (workspace defaults, project bindings, run overrides). This section defines deterministic precedence and conflict rules.

Layers (from lowest to highest precedence)
1) **Global defaults** (shipped policy registries)
2) **Workspace policy set** (workspace defaults)
3) **Project policy set** (project bindings override workspace)
4) **Run-time policy decisions** (policy hook outcomes recorded in run manifest)
5) **Approved overrides** (explicit override decisions with expiry)

Conflict rules (hard and simple)
- **Most restrictive wins** by default.
- Overrides can relax restrictions only if explicitly allowed by policy set.
- If two rules conflict and neither is strictly more restrictive, deny by default.

Examples
- If workspace allows external execution but project forbids it → forbid (project wins).
- If risk class says “verified knowledge required” but run requests “draft” → deny unless approved override exists and policy set allows it.

Piece B — Machine-readable policy set schema (where rules live)

File: `Axion/libraries/policy/schemas/policy_set.v1.schema.json`

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "schema_version": "1.0.0",
  "id": "policy_set.v1",
  "type": "object",
  "required": ["policy_set_id", "rules", "created_at", "updated_at", "owner"],
  "properties": {
    "policy_set_id": { "type": "string", "pattern": "^POLSET-[A-Z0-9]{6,}$" },

    "rules": {
      "type": "object",
      "required": ["precedence_mode", "override_permissions"],
      "properties": {
        "precedence_mode": {
          "type": "string",
          "enum": ["most_restrictive_wins", "explicit_priority"]
        },

        "explicit_priority": {
          "type": "array",
          "description": "Only used when precedence_mode==explicit_priority",
          "items": { "type": "string", "enum": ["global", "workspace", "project", "run", "override"] }
        },

        "override_permissions": {
          "type": "object",
          "description": "Which hook points can be overridden by risk class",
          "additionalProperties": {
            "type": "object",
            "properties": {
              "PROTOTYPE": { "type": "boolean" },
              "PROD": { "type": "boolean" },
              "COMPLIANCE": { "type": "boolean" }
            },
            "additionalProperties": false
          }
        },

        "max_override_duration_hours": {
          "type": "object",
          "properties": {
            "PROTOTYPE": { "type": "integer", "minimum": 1 },
            "PROD": { "type": "integer", "minimum": 1 },
            "COMPLIANCE": { "type": "integer", "minimum": 1 }
```

```
        },
        "additionalProperties": false
      },

      "deny_by_default": { "type": "boolean", "default": true }
    },
    "additionalProperties": false
  },

  "created_at": { "type": "string", "format": "date-time" },
  "updated_at": { "type": "string", "format": "date-time" },
  "owner": { "type": "string", "minLength": 2 }
 },
 "additionalProperties": false
}
```

Piece C — Conflict resolution rules (operational)

File: `Axion/libraries/policy/POL-3_conflict_rules.md`

```
---
library: policy
id: POL-3a
schema_version: 1.0.0
status: draft
---

# POL-3a — Conflict Resolution Rules

## Default: most_restrictive_wins
When two rules conflict, choose the stricter interpretation:
- external_allowed: false is stricter than true
- min_maturity: verified is stricter than reviewed is stricter than draft
- lock_mode: locked is stricter than pinned_required is stricter than open
- on_exceed: block is stricter than require_approval is stricter than warn is stricter than degrade

## If not comparable
If the engine cannot determine strictness ordering:
- deny_by_default == true → deny
- otherwise require_approval

## Overrides
```

Overrides can relax a restriction only if:
- policy_set.override_permissions[hook_point][risk_class] == true
- override expiry <= max_override_duration_hours[risk_class]

Piece D — Starter policy set (baseline)

File: `Axion/libraries/policy/registries/policy_sets.v1.json`


```
{
  "schema_version": "1.0.0",
  "registry_id": "policy_sets.v1",
  "policy_sets": [
    {
      "policy_set_id": "POLSET-BASE01",
      "rules": {
        "precedence_mode": "most_restrictive_wins",
        "override_permissions": {
          "GATE_OVERRIDE": { "PROTOTYPE": true, "PROD": true, "COMPLIANCE": false },
          "QUOTA_CHECK": { "PROTOTYPE": true, "PROD": true, "COMPLIANCE": true },
          "PIN_RESOLUTION": { "PROTOTYPE": true, "PROD": false, "COMPLIANCE": false },
          "ADAPTER_SELECTION": { "PROTOTYPE": true, "PROD": false, "COMPLIANCE": false
},
          "KIT_EXPORT": { "PROTOTYPE": true, "PROD": false, "COMPLIANCE": false }
        },
        "max_override_duration_hours": { "PROTOTYPE": 720, "PROD": 168, "COMPLIANCE": 24
},
        "deny_by_default": true
      },
      "created_at": "2026-03-05T00:00:00Z",
      "updated_at": "2026-03-05T00:00:00Z",
      "owner": "axion"
    }
  ]
}
```

Piece E — Determinism rules

File: `Axion/libraries/policy/POL-3_determinism_rules.md`

---

library: policy
id: POL-3b
schema_version: 1.0.0
status: draft

POL-3b — Determinism Rules (Precedence)

- Precedence evaluation uses a fixed layer order (unless policy_set specifies explicit_priority).
- “Most restrictive wins” uses closed strictness orderings defined in POL-3a.
- When non-comparable, deny_by_default controls deny vs require_approval.
- All policy decisions reference pinned policy_set_id and risk_class.

Piece F — Validation checklist

File: `Axion/libraries/policy/POL-3_validation_checklist.md`

---
library: policy
id: POL-3c
schema_version: 1.0.0
status: draft

POL-3c — Validation Checklist

- [ ] policy_set validates against schema
- [ ] precedence_mode is valid
- [ ] override_permissions covers known hook points
- [ ] max_override_duration_hours present for all risk classes
- [ ] strictness ordering rules exist (POL-3a)

Enforcement points

POL-4 — Enforcement points (what consults policy + required decisions)

Piece A — Enforcement points model (concept)

File: `Axion/libraries/policy/POL-4_enforcement_points.md`

```
---
library: policy
id: POL-4
schema_version: 1.0.0
status: draft
---

# POL-4 — Enforcement Points

## Purpose
Define where policy is enforced and what decisions must be produced.
(Invocation plumbing is in SYS-6; this defines *what must be decided*.)

## Enforcement points (minimum)
1) **Run start** (RUN_START)
   - allow/deny run initiation
   - enforce risk class defaults
   - enforce external executor permission (if chosen)

2) **Pin resolution** (PIN_RESOLUTION)
   - allow/deny run-level pin overrides
   - enforce lock_mode rules
   - enforce which targets must be pinned

3) **Adapter selection** (ADAPTER_SELECTION)
   - allow/deny adapter profile based on risk class and project constraints

4) **Quota check** (QUOTA_CHECK)
   - block / require approval / degrade when limits exceeded

5) **Gate override** (GATE_OVERRIDE)
   - allow/deny override requests
   - enforce expiry limits and required approver roles

6) **Kit export** (KIT_EXPORT)
```

- allow/deny exporting kits externally
- enforce no restricted content leakage (policy overlay, in addition to kit rules)

Output requirement
Each enforcement point must result in a policy decision record:
- decision_id
- hook_point
- outcome
- expiry (even for allow; could be end-of-run)
- evidence refs (policy_set_id + evaluated rule ids)

Piece B — Enforcement matrix (what is checked at each hook)

File: `Axion/libraries/policy/POL-4_enforcement_matrix.md`

---
library: policy
id: POL-4a
schema_version: 1.0.0
status: draft

POL-4a — Enforcement Matrix

Hook Point	Must Decide	Typical Inputs	Output Used By
RUN_START	allow run? risk_class ok? external ok?	workspace/project bindings, risk_class, executor_type	orchestrator start
PIN_RESOLUTION	allow pin overrides? pin targets satisfied?	pin_policy + requested pins	orchestrator pre-run
ADAPTER_SELECTION	adapter allowed?	adapter_profile + constraints + risk	executor launcher
QUOTA_CHECK	block/warn/approval/degrade	effective quota + forecast usage	orchestrator pre-run
GATE_OVERRIDE	override allowed? who must approve? expiry max?	gate_id, risk_class, policy_set	gate evaluator + UI
KIT_EXPORT	export allowed? restrictions?	target executor, kit content summary	packager/exporter

---

Piece C — Determinism rules

File: `Axion/libraries/policy/POL-4_determinism_rules.md`

```
---
library: policy
id: POL-4b
schema_version: 1.0.0
status: draft
---

# POL-4b — Determinism Rules (Enforcement)

- Every hook point produces exactly one decision record (allow/deny/require_approval/degrade).
- Decisions are derived from pinned policy_set + explicit inputs only.
- Decisions are recorded in the run manifest before the system acts on them.
- If require_approval, the run must pause and emit APPROVAL_REQUIRED.
```

Piece D — Validation checklist

File: `Axion/libraries/policy/POL-4_validation_checklist.md`

```
---
library: policy
id: POL-4c
schema_version: 1.0.0
status: draft
---

# POL-4c — Validation Checklist

- [ ] Enforcement points cover the 6 minimum hook points
- [ ] Each point requires a decision record with expiry + evidence
- [ ] require_approval triggers pause + notification
- [ ] Decisions are recorded before action is taken
```

Minimum viable policy

POL-5 — Minimum viable policy set (runtime-required files + “done”)

Piece A — MV policy inventory

File: `Axion/libraries/policy/POL-5_minimum_viable_set.md`

---
library: policy
id: POL-5
schema_version: 1.0.0
status: draft

POL-5 — Minimum Viable policy/ Set

Goal
Define the smallest policy library set required for deterministic governance.

Required runtime files (minimum)
1) Core docs
- POL-0_purpose.md
- POL-0_boundary_checklist.md
- POL-1_risk_class_model.md
- POL-1_determinism_rules.md
- POL-1_validation_checklist.md
- POL-2_override_policy_model.md
- POL-2_override_rules.md
- POL-2_validation_checklist.md
- POL-3_precedence_model.md
- POL-3_conflict_rules.md
- POL-3_determinism_rules.md
- POL-3_validation_checklist.md
- POL-4_enforcement_points.md
- POL-4_enforcement_matrix.md
- POL-4_determinism_rules.md
- POL-4_validation_checklist.md
- POL-5_minimum_viable_set.md

2) Schemas (must be in schema_registry)
- schemas/risk_classes.v1.schema.json
- schemas/policy_set.v1.schema.json

- schemas/override_request.v1.schema.json
- schemas/override_decision.v1.schema.json

3) Registries (runtime defaults)
- registries/risk_classes.v1.json
- registries/policy_sets.v1.json

Piece B — Definition of done (policy/)

File: `Axion/libraries/policy/POL-5_definition_of_done.md`

---
library: policy
id: POL-5a
schema_version: 1.0.0
status: draft

POL-5a — policy/ Definition of Done

policy/ is “done” when:

Schemas + registries
- [ ] All policy schemas validate (JSON Schema check)
- [ ] risk_classes registry validates and covers PROTOTYPE/PROD/COMPLIANCE
- [ ] policy_sets registry validates and includes at least one baseline set

Runtime behavior (contract-level)
- [ ] Policy engine can resolve effective policy via precedence rules (POL-3)
- [ ] Each enforcement hook point produces a decision record (POL-4)
- [ ] Override requests require reason + approver + expiry
- [ ] Override duration limits enforced per risk class

Determinism
- [ ] Decisions are a pure function of pinned policy_set + explicit request context
- [ ] Non-comparable conflicts resolve via deny_by_default (or require_approval when configured)
- [ ] All decisions are recorded in run manifest before actions are taken

---

Piece C — Minimal folder tree

File: `Axion/libraries/policy/POL-5_minimal_tree.txt`

```
Axion/libraries/policy/
  POL-0_purpose.md
  POL-0_boundary_checklist.md
  POL-1_risk_class_model.md
  POL-1_determinism_rules.md
  POL-1_validation_checklist.md
  POL-2_override_policy_model.md
  POL-2_override_rules.md
  POL-2_validation_checklist.md
  POL-3_precedence_model.md
  POL-3_conflict_rules.md
  POL-3_determinism_rules.md
  POL-3_validation_checklist.md
  POL-4_enforcement_points.md
  POL-4_enforcement_matrix.md
  POL-4_determinism_rules.md
  POL-4_validation_checklist.md
  POL-5_minimum_viable_set.md
  POL-5_definition_of_done.md
  POL-5_minimal_tree.txt

  schemas/
    risk_classes.v1.schema.json
    policy_set.v1.schema.json
    override_request.v1.schema.json
    override_decision.v1.schema.json

  registries/
    risk_classes.v1.json
    policy_sets.v1.json
```

Say **“next”** to start the next library folder. Recommended next is **intake/** (INT-0).